

=====

Problem: Anagram

Level: Medium

Source: Digital/Anagram.java

Time complexity: $O(n)$ average caseSpace complexity: $O(n)$

Your original code:

```
import java.util.*;

public class Anagram{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str1 = sc.nextLine();
        char[] ch1 = str1.toCharArray();
        String str2 = sc.nextLine();
        char[] ch2 = str2.toCharArray();

        if(str1.length() != str2.length()){
            System.out.print(false);
            return;
        }

        HashMap<Character,Integer> map1 = new HashMap<>();
        HashMap<Character,Integer> map2 = new HashMap<>();

        for(Character ch : ch1){
            if(map1.containsKey(ch)){
                map1.put(ch, map1.get(ch)+1);
            }
            else{
                map1.put(ch,1);
            }
        }

        for(Character ch : ch2){
            if(map2.containsKey(ch)){
                map2.put(ch, map2.get(ch)+1);
            }
            else{
                map2.put(ch,1);
            }
        }

        System.out.print(map1.equals(map2));
    }
}
```

=====

Problem: Best Time to Buy Sell

Level: Medium

Source: Digital/Best_Time_to_Buy_Sell.java

Time complexity: $O(n)$ Space complexity: $O(n)$

Your original code:

```
import java.util.Scanner;

public class Best_Time_to_Buy_Sell{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
    }
}
```

```

    }

    int minPrice = arr[0];
    int maxProfit = 0;

    for(int i=1;i<n;i++){
        if(arr[i] < minPrice){
            minPrice = arr[i];
        }

        maxProfit = Math.max(maxProfit, arr[i]-minPrice);
    }

    System.out.print("Best time to buy: " + minPrice + " " + "Best time to Sell: " + maxProfit);
}
}

```

```

=====
Problem: First Non Repeating Char
Level: Medium
Source: Digital/First_Non_Repeating_Char.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class First_Non_Repeating_Char{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        HashMap<Character,Integer> map = new HashMap<>();

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);

            if(map.containsKey(ch)){
                map.put(ch, map.getOrDefault(ch,0)+1);
            }else{
                map.put(ch,1);
            }
        }

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);
            if(map.get(ch) == 1){
                System.out.print("First Repeating char is: " + ch);
                return;
            }
        }

        System.out.print("Not found");

    }
}

```

```

=====
Problem: Longest Sequence
Level: Medium
Source: Digital/Longest_Sequence.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class Longest_Sequence{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
    }
}

```

```

    int n = sc.nextInt();
    int[] arr = new int[n];

    for(int i=0;i<n;i++){
        arr[i] = sc.nextInt();
    }

    HashSet<Integer> set = new HashSet<>();

    for(int num : arr){
        set.add(num);
    }

    int maxLength = 0;
    for(int num : set){
        if(!set.contains(num-1)){
            int currentNum = num;
            int length = 1;

            while(set.contains(currentNum+1)){
                currentNum++;
                length++;
            }
            maxLength = Math.max(maxLength, length);
        }
    }

    System.out.print("Longest Sequence length is: " + maxLength);

}
}

```

```

=====
Problem: Longest Subarray Sum
Level: Medium
Source: Digital/Longest_Subarray_Sum.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class Longest_Subarray_Sum{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        int target = sc.nextInt();

        for(int num : arr){
            int res = target - num;
            if(map.containsKey(res)){
                System.out.print(num + " " + res);
                return;
            }
            map.put(num,1);
        }
    }
}

```

```

=====

```

Problem: Longest Subarray With Sum Less than or Equal to K

Level: Medium

Source: Digital/Longest_Subarray_WithSumLess than or Equal to K.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```
import java.util.*;

public class Longest_Subarray_WithSumLess than or Equal to K {

    public static void main(String[] args) {

        int n = 5;
        int[] arr = {2, -1, 3, -2, 4};
        int k = 5;

        int[] prefix = new int[n + 1];

        // Build prefix sum
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + arr[i];
        }

        // Build prefix maximum
        int[] prefixMax = new int[n + 1];
        prefixMax[0] = prefix[0];

        for (int i = 1; i <= n; i++) {
            prefixMax[i] = Math.max(prefixMax[i - 1], prefix[i]);
        }

        int maxLen = 0;

        for (int i = 0; i <= n; i++) {

            int target = prefix[i] - k;

            // Binary search: first prefixMax[j] >= target
            int left = 0;
            int right = i;

            while (left <= right) {

                int mid = left + (right - left) / 2;

                if (prefixMax[mid] >= target) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            }

            // left is the earliest valid j
            if (left <= i) {
                int len = i - left;
                maxLen = Math.max(maxLen, len);
            }
        }

        System.out.println(maxLen);
    }
}
```

=====

Problem: Longest Subarray sum Equals K

Level: Medium

Source: Digital/Longest_Subarray_sumEqualsK.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```
import java.util.*;
```

```
public class Longest_Subarray_sumEqlsK{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int[] arr = {1,2,-2,2,3};
        int k = sc.nextInt();

        HashMap<Integer,Integer> map = new HashMap<>();

        int sum = 0;
        int maxLen = 0;

        for(int i=0;i<arr.length;i++){
            sum = sum+arr[i];

            if(sum == k){
                maxLen = i+1;
            }

            if(map.containsKey(sum-k)){
                int len = i - map.get(sum - k);//finds start of subarray

                maxLen = Math.max(maxLen,len);//compares maxlen and len value.
            }

            if(!map.containsKey(sum-k)){
                map.put(sum,i);
            }
        }

        System.out.print(maxLen);
    }
}
```

```
=====
```

Problem: Majority Element

Level: Medium

Source: Digital/Majority_Element.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```
import java.util.*;

public class Majority_Element{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            if(map.containsKey(num)){
                map.put(num, map.getDefault(num,0)+1);
            }else{
                map.put(num,1);
            }
        }

        int majority = n/2;

        for(int num : map.keySet()){
            if(map.get(num) > majority){
                System.out.println(num);
            }
        }
    }
}
```

```

        return;
    }
}

System.out.print("Not Found");

}
}

```

```

=====
Problem: Max Consecutive 1s
Level: Medium
Source: Digital/Max_Consecutive_1s.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;

public class Max_Consecutive_1s{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int currentCount = 0;

        int maxCount = 0;
        for(int i=0;i<n;i++){
            if(arr[i] == 1){
                currentCount++;
                maxCount = Math.max(maxCount,currentCount);
            }else{
                currentCount = 0;
            }
        }

        System.out.print(maxCount);
    }
}

```

```

=====
Problem: Max Sum Of Subarray Of Ksize
Level: Medium
Source: Digital/Max_SumOfSubarray_OfKsize.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class Max_SumOfSubarray_OfKsize{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
        int k = sc.nextInt();

        int windowSum = 0;

        for(int i=0;i<k;i++){
            windowSum = windowSum+arr[i];
        }
    }
}

```

```

    }

    int max_sum = windowSum;

    for(int i=k;i<n;i++){
        windowSum = windowSum - arr[i-k]+arr[i];
        if(windowSum > max_sum){
            max_sum = windowSum;
        }
    }
    System.out.print(max_sum);
}
}

```

=====

Problem: Max Sum Subarray Of Any Size

Level: Medium

Source: Digital/Max_Sum_Subarray_OfAnySize.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;

public class Max_Sum_Subarray_OfAnySize{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int currentSum = arr[0];
        int maxSum = arr[0];

        for(int i=1;i<n;i++){
            if(arr[i] > currentSum+arr[i]){
                currentSum = arr[i];
            }
            else{
                currentSum += arr[i];
            }

            maxSum = Math.max(maxSum,currentSum);
        }
        System.out.println(maxSum);
        sc.close();
    }
}

```

=====

Problem: Merge Sorted Arrays

Level: Medium

Source: Digital/Merge_Sorted_Arrays.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Merge_Sorted_Arrays{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[] arr = new int[n];
    }
}

```

```

int[] arr2 = new int[m];

for(int i=0;i<n;i++){
    arr[i] = sc.nextInt();
}
for(int i=0;i<m;i++){
    arr2[i] = sc.nextInt();
}

int k = 0;
int[] result = new int[n+m];

int i=0;
int j=0;

while(i<arr.length && j<arr2.length){
    if(arr[i] < arr2[j]){
        result[k] = arr[i];
        i++;
        k++;
    }else if(arr2[j] < arr[i]){
        result[k] = arr2[j];
        j++;
        k++;
    }else {
        result[k] = arr[i];
        k++;
        result[k] = arr2[j];
        k++;
    }
}

while(i < arr.length){
    result[k] = arr[i];
    i++;
    k++;
}

while(j < arr2.length){
    result[k] = arr2[j];
    j++;
    k++;
}

int h = n+m;
for(int e=0;e<h;e++){
    System.out.print(result[e] + " ");
}
}

```

=====

Problem: Missing Number

Level: Medium

Source: Digital/Missing_Number.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;

public class Missing_Number{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
        int Expsum =0;
    }
}

```

```

    int Actsum = 0;
    int missing = n;

    for(int i=0;i<n;i++){
        // Expsum = Expsum+i+1;
        // Actsum = Actsum+arr[i];

        // missing = Expsum - Actsum;

        //xor operation
        missing = missing ^ i ^ arr[i];
    }
    System.out.print(missing);
}
}

```

=====

Problem: Move Zeroes

Level: Medium

Source: Digital/Move_Zeroes.java

Time complexity: O(n)

Space complexity: O(n)

Your original code:

```

import java.util.Scanner;

public class Move_Zeroes{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int j = 0;

        for(int i=0;i<n;i++){
            if(arr[i] != 0){
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                j++;
            }
        }

        for(int i=0;i<n;i++){
            System.out.print(arr[i] + " ");
        }
    }
}

```

=====

Problem: Prefix Sum Of Subarray

Level: Medium

Source: Digital/Prefix_SumOf_Subarray.java

Time complexity: O(n) average case

Space complexity: O(n)

Your original code:

```

// import java.util.*;

// public class Prefix_SumOf_Subarray{
//     public static void main(String[] args){
//         Scanner sc = new Scanner(System.in);

//         int n = sc.nextInt();
//         int[] arr = new int[n];
//     }
// }

```

```

//      for(int i=0;i<n;i++){
//          arr[i] = sc.nextInt();
//      }

//      int k = sc.nextInt();
//      int sum = 0;
//      int count = 0;
//      HashMap<Integer,Integer> map = new HashMap<>();
//      map.put(0,1);

//      for(int x=0;x<n;x++){
//          sum += arr[x];
//          if(map.containsKey(sum-k)){
//              count = count+map.get(sum-k);
//          }
//          else{
//              map.put(sum, map.getOrDefault(sum,0)+1);
//          }
//      }

//      System.out.print(count);
//      // System.out.print(map);

//  }
// }

import java.util.*;

public class Prefix_SumOf_Subarray {

    public static void main(String[] args) {

        int[] arr = {1, -1, 1, 1, -1};
        int k = 1;

        int sum = 0;
        int count = 0;

        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);

        for (int i = 0; i < arr.length; i++) {

            sum += arr[i];

            System.out.println("\nIndex: " + i);
            System.out.println("Element: " + arr[i]);
            System.out.println("Current Sum: " + sum);

            int required = sum - k;

            System.out.println("sum - k = " + required);

            if (map.containsKey(required)) {

                count += map.get(required);

                System.out.println("Found " + required + " in map");
                System.out.println("Frequency: " + map.get(required));
                System.out.println("Count: " + count);

            } else {

                System.out.println(required + " is NOT in map");
            }

            map.put(sum, map.getOrDefault(sum, 0) + 1);

            System.out.println("Map: " + map);
        }

        System.out.println("\nFinal Answer: " + count);
    }
}

```

```
    }
}
```

```
=====
Problem: Remove Duplicates In Sorted Array
```

```
Level: Medium
```

```
Source: Digital/Remove_DuplicatesIn_SortedArray.java
```

```
Time complexity: O(n)
```

```
Space complexity: O(n)
```

```
Your original code:
```

```
import java.util.Scanner;

public class Remove_DuplicatesIn_SortedArray{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int j = 0;

        for(int i=1;i<n;i++){
            if(arr[i] != arr[j]){
                j++;
                arr[j] = arr[i];
            }
        }

        for(int i=0;i<=j;i++){
            System.out.print(arr[i] + " ");
        }

        System.out.println();
        System.out.println("unique element: " + (j + 1));
    }
}
```

```
=====
Problem: String Palindrome
```

```
Level: Medium
```

```
Source: Digital/String_Palindrome.java
```

```
Time complexity: O(n)
```

```
Space complexity: O(1) auxiliary space
```

```
Your original code:
```

```
import java.util.Scanner;

public class String_Palindrome{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();

        str = str.replace(" ", "").toLowerCase();

        int left = 0;
        int right = str.length()-1;
        boolean palindrome = true;

        while(left < right){
            if(str.charAt(left) == str.charAt(right)){
                palindrome = false;
            }
            left++;
            right--;
        }
    }
}
```

```

        if(palindrome){
            System.out.print("Not Palindrome");
        }else{
            System.out.print("Palindrome");
        }
    }
}

```

```

=====
Problem: Sub Array Sum Equals K
Level: Medium
Source: Digital/SubArray_Sum_EqualsK.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class SubArray_Sum_EqualsK{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int k = sc.nextInt();

        int prefixSum = 0;
        int count = 0;

        HashMap<Integer,Integer> map = new HashMap<>();
        map.put(0,1);
        for(int i=0;i<n;i++){
            prefixSum = prefixSum+arr[i];

            if(map.containsKey(prefixSum - k)){
                count = count + map.get(prefixSum - k);
            }

            map.put(prefixSum, map.getOrDefault(prefixSum,0)+1);
        }

        System.out.println(count);
        System.out.print(map);
    }
}

```

```

=====
Problem: Subarr With Eql No Ofzeroandone
Level: Medium
Source: Digital/Subarr_With_EqlNoOfzeroandone.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class Subarr_With_EqlNoOfzeroandone{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
    }
}

```

```

        int maxLen = 0;

        HashMap<Integer,Integer> map = new HashMap<>();
        int difference = 0;
        map.put(0, -1);

        for(int i=0;i<n;i++){
            if(arr[i] == 0){
                difference--;
            }
            if(arr[i] == 1){
                difference++;
            }

            if(map.containsKey(difference)){
                int len = i-map.get(difference);
                maxLen = Math.max(maxLen, len);
            }else
                map.put(difference,i);
        }
        System.out.println(maxLen);
        System.out.print(map);
    }
}

```

```

=====
Problem: Subarray Sum Div BYK
Level: Medium
Source: Digital/SubarraySum_Div_BYK.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class SubarraySum_Div_BYK{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int k =sc.nextInt();
        int sum = 0;
        int count = 0;

        HashMap<Integer,Integer> map = new HashMap<>();

        map.put(0,1);

        for(int i=0;i<n;i++){
            sum = sum+arr[i];
            int remainder = sum%k;

            if(map.containsKey(remainder)){
                count = count+map.get(remainder);
            }
            map.put(remainder,map.getDefault(remainder,0)+1);
        }

        System.out.print(count);
        System.out.print(map);
    }
}

```

```
=====
Problem: Anagram
Level: Medium
Source: HashMap/Anagram.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:
```

```
import java.util.*;
public class Anagram{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str1 = sc.nextLine();
        char[] ch1 = str1.toCharArray();
        String str2 = sc.nextLine();
        char[] ch2 = str2.toCharArray();

        HashMap<Character,Integer> map1 = new HashMap<>();
        HashMap<Character,Integer> map2 = new HashMap<>();

        for(Character ch : ch1){
            if(map1.containsKey(ch)){
                map1.put(ch ,map1.get(ch)+1);
            }else{
                map1.put(ch,1);
            }
        }
        for(Character ch : ch2){
            if(map2.containsKey(ch)){
                map2.put(ch,map2.get(ch)+1);
            }else{
                map2.put(ch,1);
            }
        }

        // String res = map1.equals(map2) ? "true" : "false";
        System.out.println(map1.equals(map2));

    }
}
```

```
=====
Problem: Count distinct elmnts
Level: Medium
Source: HashMap/Count_distinct_elmnts.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:
```

```
import java.util.*;
public class Count_distinct_elmnts{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            map.put(num,1);//HashMap does not allow duplicate elements
        }
        System.out.println(map.size());//so it will print unique element array size.
    }
}
```

```
=====
Problem: Duplicate Elmnt
Level: Medium
Source: HashMap/Duplicate_Elmnt.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:
```

```
import java.util.*;
public class Duplicate_Elmnt{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap <Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            if(map.containsKey(num)){
                map.put(num,map.get(num)+1);
            }else{
                map.put(num,1);
            }
        }

        for(int key : map.keySet()){
            if(map.get(key) > 1){
                System.out.println(key);
            }
        }
    }
}
```

```
=====
Problem: First Non Reapeating Character
Level: Medium
Source: HashMap/First_Non_Reapeating_Character.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:
```

```
import java.util.*;

public class First_Non_Reapeating_Character {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        char[] ch = str.toCharArray();
        boolean found = false;

        HashMap<Character, Integer> map = new HashMap<>();

        for (char cha : ch) {
            if (map.containsKey(cha)) {
                map.put(cha, map.get(cha) + 1);
            } else {
                map.put(cha, 1);
            }
        }

        for (char key : ch) {
            if (map.get(key) == 1) {
                found = true;
                System.out.println(key);
            }
        }
    }
}
```

```

        break;
    }
}
if (!found) {
    System.out.println("Not Found");
}
}
}
}

```

=====

Problem: Frequency Of Character

Level: Medium

Source: HashMap/FrequencyOfCharacter.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class FrequencyOfCharacter{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        char[] ch = str.toCharArray();
        int max = 0;
        char maxchar = ' ';

        HashMap<Character,Integer> map = new HashMap<>();

        for(char cha : ch){
            if(map.containsKey(cha)){
                map.put(cha, map.get(cha)+1);
            }else{
                map.put(cha,1);
            }
            // if(map.get(cha) > max){
            //     max = map.get(cha);
            // }

            // OR
            max = Math.max(max,map.get(cha)); //prints maximum frequency of character in a string
            maxchar = (char) Math.max(maxchar, cha); // prints character with highest ascii value.
        }

        System.out.println(map); //prints number of occurrence of each character
        System.out.println(max); //prints maximum repeated character in value
        System.out.println(maxchar); //prints character with maximum ascii value.
    }
}

```

=====

Problem: Frequency Of Numbers

Level: Medium

Source: HashMap/FrequencyOfNumbers.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class FrequencyOfNumbers {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int max = 0;
    }
}

```

```

int[] arr = {2,2,2,3,3,2};
HashMap<Integer,Integer> map = new HashMap<>();
for(int num : arr){
    if(map.containsKey(num)){
        map.put(num,map.get(num)+1);
    }else{
        map.put(num,1);
    }

    max = Math.max(max,map.get(num));

}

System.out.println(map);
System.out.println(max);

}
}

```

=====

Problem: Frequency Of Words

Level: Medium

Source: HashMap/FrequencyOfWords.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class FrequencyOfWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        int max = 0;

        HashMap<String, Integer> map = new HashMap<>();
        String[] split = str.split(" ");

        for (String word : split) {
            if (map.containsKey(word)) {
                map.put(word, map.get(word) + 1);
            } else {
                map.put(word, 1);
            }
        }
        for (String key : map.keySet()) {
            max = Math.max(max, map.get(key));
        }
        System.out.println(map);

        System.out.println(max);

    }
}

```

=====

Problem: Frequency Using Hashmap

Level: Medium

Source: HashMap/FrequencyUsingHashmap.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class FrequencyUsingHashmap{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
    }
}

```

```

    int n = sc.nextInt();
    int[] arr = new int[n];

    for(int i=0;i<n;i++){
        arr[i] = sc.nextInt();
    }

    HashMap<Integer,Integer> map = new HashMap<>();

    for(int num : arr){
        if(map.containsKey(num)){
            map.put(num, map.get(num)+1);
        }else{
            map.put(num,1);
        }
    }

    System.out.println(map);
}
}

```

=====

Problem: Frequent Element

Level: Medium

Source: HashMap/FrequentElement.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;
public class FrequentElement{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        int max = 0;
        int maxEl = 0;

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            if(map.containsKey(num)){
                map.put(num, map.get(num)+1);

            }else
                map.put(num,1);

        }

        for(int key : map.keySet()){
            if(map.get(key) > max){
                max = map.get(key);
                maxEl = key;
            }
        }
        System.out.println(map);
        System.out.println(max);
        System.out.println(maxEl);
    }
}

```

=====

Problem: Majority Element

Level: Medium

Source: HashMap/Majority_Element.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```
import java.util.*;

public class Majority_Element{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            if(map.containsKey(num)){
                map.put(num,map.get(num)+1);
            }else{
                map.put(num,1);
            }
        }
        int max = 0;
        int maxEl = 0;
        for(int key : map.keySet()){
            if(map.get(key) > n/2){
                System.out.println(key);
                return;
            }
        }

        System.out.println(max);
        System.out.println(maxEl);
    }
}
```

=====

Problem: Top K Frequent Elmnts

Level: Medium

Source: HashMap/TopK_Frequent_Elmnts.java

Time complexity: $O(n \log k)$ in the common Top-K/heap approach

Space complexity: $O(n)$

Your original code:

```
import java.util.*;

public class TopK_Frequent_Elmnts {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int k = 2;

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int num : arr) {
            if (map.containsKey(num)) {
                map.put(num, map.get(num) + 1);
            } else {
                map.put(num, 1);
            }
        }

        PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>((a, b) -> a.getValue() -
b.getValue());
```

```

        for (Map.Entry<Integer,Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " " + entry.getValue());
            pq.offer(entry);
            if (pq.size() > k) {
                pq.poll();
            }
        }

        while(!pq.isEmpty()){
            System.out.println(pq.poll().getKey());
        }

    }
}

```

=====

Problem: Two Sum

Level: Medium

Source: HashMap/Two_Sum.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;
public class Two_Sum{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
        int target = sc.nextInt();
        boolean found = false;

        HashMap <Integer,Integer> map = new HashMap<>();

        int res = 0;

        for(int i=0;i<n;i++){
            res = target - arr[i];
            if(map.containsKey(res)){
                found = true;
                System.out.println(map.get(res) + " " + i);
            }else{
                map.put(arr[i],i);
            }
        }

    }
}

```

=====

Problem: Kth Largest Element

Level: Medium

Source: Heap/Kth_Largest_Element.java

Time complexity: $O(n \log k)$ in the common Top-K/heap approach

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Kth_Largest_Element{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int[] arr = {3,2,1,5,6,4};
        int k = 2;
    }
}

```

```

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for(int num : arr){
            pq.add(num);

            if(pq.size() > k){
                pq.poll();
            }
        }

        System.out.println(pq.peek());
    }
}

```

```

=====
Problem: Max Heap Demo
Level: Medium
Source: Heap/MaxHeapDemo.java
Time complexity:  $O(n \log k)$  in the common Top-K/heap approach
Space complexity:  $O(n)$ 
Your original code:

```

```

import java.util.*;

public class MaxHeapDemo{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        maxHeap.add(30);
        maxHeap.add(40);
        maxHeap.add(50);
        maxHeap.add(10);

        System.out.println(maxHeap.peek());
        System.out.println(maxHeap.poll());
        System.out.println(maxHeap.peek());

    }
}

```

```

=====
Problem: Min Heap Demo
Level: Medium
Source: Heap/MinHeapDemo.java
Time complexity:  $O(n \log k)$  in the common Top-K/heap approach
Space complexity:  $O(n)$ 
Your original code:

```

```

import java.util.*;

public class MinHeapDemo{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        pq.add(40);
        pq.add(30);
        pq.add(5);
        pq.add(15);

        System.out.println(pq);
        System.out.println(pq.peek());

        pq.poll();

        System.out.println(pq.peek());
    }
}

```

```
    }
}
```

```
//output : [5,15,30,40]
```

```
=====
Problem: Top K Freq Elements
Level: Medium
Source: Heap/TopK_Freq_Elements.java
Time complexity:  $O(n \log k)$  in the common Top-K/heap approach
Space complexity:  $O(n)$ 
Your original code:
```

```
import java.util.*;

public class    {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] arr = {1, 1, 1, 2, 2, 5, 5, 5, 3};
        int k = 2;

        HashMap<Integer, Integer> hmap = new HashMap<>();
        PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>((a, b) -> a.getValue() -
b.getValue());

        for (int num : arr) {
            hmap.put(num, hmap.getDefault(num, 0) + 1);
        }
        System.out.println(hmap);

        for (Map.Entry<Integer, Integer> entry : hmap.entrySet()) {
            System.out.println(entry.getKey() + " " + entry.getValue());

            pq.offer(entry);

            if (pq.size() > k) {
                pq.poll();
            }

        }
        while (!pq.isEmpty()) {
            System.out.println(pq.poll().getKey());
        }
    }
}
```

```
=====
Problem: Best time To Buy Sell
Level: Medium
Source: zoho/Best_timeToBuySell.java
Time complexity:  $O(n)$ 
Space complexity:  $O(n)$ 
Your original code:
```

```
import java.util.*;

public class Best_timeToBuySell{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }
    }
}
```

```

int minPrice = arr[0];
int maxProfit = 0;

int tempBuyDay = 0;
int buyDay = 0;
int sellDay = 0;

for(int i=1;i<n;i++){
    if(arr[i] < minPrice){
        minPrice = arr[i];
        tempBuyDay = i;
    }

    int profit = arr[i] - minPrice;
    if(profit > maxProfit){
        maxProfit = profit;
        buyDay = tempBuyDay;
        sellDay = i;
    }
}

System.out.println("Best time to Buy at a day is: " + buyDay);
System.out.println("Best time to sell at a Day is: " + sellDay);
}
}

```

=====

Problem: Count distinct elmnts

Level: Medium

Source: zoho/Count_distinct_elmnts.java

Time complexity: O(n) average case

Space complexity: O(n)

Your original code:

```

import java.util.*;
public class Count_distinct_elmnts{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            map.put(num,1);//HashMap does not allow duplicate elements
        }
        System.out.println(map.size());//so it will print unique element array size.
    }
}

```

=====

Problem: Duplicate Elmnt

Level: Medium

Source: zoho/Duplicate_Elmnt.java

Time complexity: O(n) average case

Space complexity: O(n)

Your original code:

```

import java.util.*;
public class Duplicate_Elmnt{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){

```

```

        arr[i] = sc.nextInt();
    }

    HashMap <Integer,Integer> map = new HashMap<>();

    for(int num : arr){
        if(map.containsKey(num)){
            map.put(num,map.get(num)+1);
        }else{
            map.put(num,1);
        }
    }

    for(int key : map.keySet()){
        if(map.get(key) > 1){
            System.out.println(key);
        }
    }
}

```

```

=====
Problem: First Non Reapeating Character
Level: Medium
Source: zoho/First_Non_Reapeating_Character.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```

import java.util.*;

public class First_Non_Reapeating_Character {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        char[] ch = str.toCharArray();
        boolean found = false;

        HashMap<Character, Integer> map = new HashMap<>();

        for (char cha : ch) {
            if (map.containsKey(cha)) {
                map.put(cha, map.get(cha) + 1);
            } else {
                map.put(cha, 1);
            }
        }

        for (char key : ch) {
            if (map.get(key) == 1) {
                found = true;
                System.out.println(key);
                break;
            }
        }
        if (!found) {
            System.out.println("Not Found");
        }
    }
}

```

```

=====
Problem: Frequency Of Numbers
Level: Medium
Source: zoho/FrequencyOfNumbers.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:

```

```
import java.util.*;

public class FrequencyOfNumbers {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int max = 0;

        int[] arr = {2,2,2,3,3,2};
        HashMap<Integer,Integer> map = new HashMap<>();
        for(int num : arr){
            if(map.containsKey(num)){
                map.put(num,map.get(num)+1);
            }else{
                map.put(num,1);
            }

            max = Math.max(max,map.get(num));
        }

        System.out.println(map);
        System.out.println(max);
    }
}
```

```
=====
Problem: Longest Consecutive Seqnce
Level: Medium
Source: zoho/Longest_Consecutive_Seqnce.java
Time complexity: O(n) average case
Space complexity: O(n)
Your original code:
```

```
import java.util.*;

public class Longest_Consecutive_Seqnce{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        System.out.println("Longest consecutive sequence lenght is : " + longestConsecutive(arr) );
    }

    public static int longestConsecutive(int[] nums){

        HashSet<Integer> set = new HashSet<>();
        for(int num : nums){
            set.add(num);
        }

        int longest = 0;

        for(int num : set){
            if(!set.contains(num-1)){
                int currentNum = num;
                int count = 1;
                while(set.contains(currentNum + 1)){
                    currentNum++;
                    count++;
                }
                longest = Math.max(longest, count);
            }
        }
    }
}
```

```

    }
    return longest;
}
}

```

=====

Problem: Majority Element

Level: Medium

Source: zoho/Majority_Element.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Majority_Element{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int num : arr){
            if(map.containsKey(num)){
                map.put(num,map.get(num)+1);
            }else{
                map.put(num,1);
            }
        }
        int max = 0;
        int maxEl = 0;
        for(int key : map.keySet()){
            if(map.get(key) > n/2){
                System.out.println(key);
                return;
            }
        }

        System.out.println(max);
        System.out.println(maxEl);
    }
}

```

=====

Problem: Max Subarray Sum

Level: Medium

Source: zoho/Max_Subarray_Sum.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Max_Subarray_Sum{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int Final_sum = max_subarray_sum(arr);
    }
}

```

```

        System.out.println("Maximum sub array sum is: " + Final_sum);

    }

    public static int max_subarray_sum(int[] nums){
        int currentSum = nums[0];
        int maxSum = nums[0];
        int tempStart = 0;
        int start = 0;
        int end = 0;

        for(int i=1;i<nums.length;i++){

            if(nums[i] > currentSum + nums[i]){
                currentSum = nums[i];
                tempStart = i;
            }
            else{
                currentSum = currentSum + nums[i];
            }

            if(currentSum > maxSum){
                maxSum = currentSum;
                start = tempStart;
                end = i;
            }

        }

        for(int j=start;j<=end;j++){
            System.out.print(nums[j] + " ");
        }

        return maxSum;
    }
}

```

=====

Problem: Merge Intervals

Level: Medium

Source: zoho/Merge_Intervals.java

Time complexity: $O(n \log n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Merge_Intervals{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[][] intervals = new int[n][2];

        for(int i=0;i<n;i++){
            for(int j=0;j<2;j++){
                intervals[i][j] = sc.nextInt();
            }
        }

        Arrays.sort(intervals, (a,b) -> a[0] - b[0]); //sorts the array which compares two intervals at
        a time
        //if result < 0 --> a comes before b || result > 0 --> b comes before a || result == 0 then both
        are considered equal in sort order

        List<int[]> result = new ArrayList<>();

        result.add(intervals[0]);

        for(int i=1;i<intervals.length;i++){

```

```

    int[] current = intervals[i];
    int[] last = result.get(result.size()-1);

    if(current[0] <= last[1]){
        last[1] = Math.max(last[1],current[1]);
    }else{
        result.add(current);
    }

}

int[][] ans = new int[result.size()][2];

for(int i=0;i<result.size();i++){
    ans[i] = result.get(i);
}

for(int i=0;i<ans.length;i++){
    System.out.println(ans[i][0] + " " + ans[i][1]);
}

}
}

```

=====

Problem: Mirror Pattern

Level: Medium

Source: zoho/Mirror_Pattern.java

Time complexity: $O(n^2)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.Scanner;

public class Mirror_Pattern {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        // for(int i=0;i<n;i++){
        //     int num = n-i;
        //     for(int j=0;j<=i;j++){
        //         System.out.print(num);
        //         System.out.print(0);
        //         num++;
        //     }
        //     System.out.println();
        // }
        // int num = 1;
        // for(int i=0;i<n;i++){
        //     for(int j=0;j<=i;j++){
        //         System.out.print(num);
        //     }
        //     System.out.println();
        //     num++;
        // }
        // for(int i=0;i<n;i++){
        //     for(int j=1;j<=n-i;j++){
        //         System.out.print(j);
        //     }
        //     System.out.println("");
        // }
        // for(int i=1;i<=n;i++){
        //     for(int j=1;j<=i;j++){
        //         System.out.print(i);
        //     }
        //     System.out.println("");
        // }
    }
}

```

```

// for (int i = 1; i <= n; i++) {
//     for (int s = 1; s <= n - i; s++) {
//         System.out.print(" ");
//     }
//     for (int k = 1; k <= i; k++) {
//         System.out.print(k);
//     }
//     for(int j=i-1;j>=1;j--)
//         System.out.print(j);
//         // num++;
//     System.out.println();
// }
for (int i = 1; i <= 5; i++) {
    for(int s = 1; s<=n-i;s++){
        System.out.print(" ");
    }

    if (i == 1) {
        System.out.println(0);
    }else {

        for (int k = n - i + 2; k <= 5; k++) {
            System.out.print(k);
        }

        System.out.print(0);

        for (int j = 5; j >= n - i + 2; j--) {
            System.out.print(j);
        }
        System.out.println("");
    }

}

}
}

```

=====

Problem: Move Zeroes

Level: Medium

Source: zoho/Move_Zeroes.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;

public class Move_Zeroes{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int j = 0;//it points to next position
        for(int i=0;i<n;i++){//it points to current position
            if(arr[i] != 0){
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                j++;
            }
        }
    }
}

```

```

    for(int index : arr){
        System.out.print(index + " ");
    }
}
}

```

=====

Problem: Pattern

Level: Medium

Source: zoho/Pattern.java

Time complexity: $O(n^2)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

import java.util.Scanner;

public class Pattern{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for(int i=1;i<=n;i++){
            for(int j=1;j<=i;j++){
                System.out.print(j);

            }
            System.out.println();
        }
    }
}

```

=====

Problem: Product Of arr Expt Self

Level: Medium

Source: zoho/Product_Of_arr_Expt_Self.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.Scanner;

public class Product_Of_arr_Expt_Self {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int[] left = new int[n];
        int[] right = new int[n];

        left[0] = 1;
        right[n - 1] = 1;

        for (int i = 1; i < n; i++) {
            left[i] = left[i - 1] * arr[i - 1];
        }
        for (int i = n - 2; i >= 0; i--) {
            right[i] = right[i + 1] * arr[i + 1];
        }

        int[] ans = new int[n];
        System.out.print("Product of array Except Self is: ");
    }
}

```

```

    for (int i = 0; i < n; i++) {
        ans[i] = left[i] * right[i];
        System.out.print(ans[i] + " ");
    }

}
}

```

=====

Problem: Remove Duplicates

Level: Medium

Source: zoho/Remove_Duplicates.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Remove_Duplicates {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.next();

        // for(int i=0;i<str.length();i++){
        //     boolean duplicate = false;
        //     for(int j=0;j<i;j++){
        //         if(str.charAt(i) == str.charAt(j)){
        //             duplicate = true;
        //             break;
        //         }
        //     }
        //     if(!duplicate){
        //         System.out.print(str.charAt(i)); // complexity =  $O(n^2)$ 
        //     }
        // }

        // int[] freq = new int[26];

        // for(int i=0;i<str.length();i++){
        //     char ch = str.charAt(i);
        //     if(freq[ch - 'a'] == 0){
        //         System.out.print(ch); // complexity =  $O(n)$ , space =  $O(1)$ 
        //         freq[ch - 'a']++;
        //     }
        // }

        HashSet<Character> hs = new HashSet<>();

        for(int i=0;i<str.length();i++){
            char ch = str.charAt(i);

            if(!hs.contains(ch)){
                System.out.print(ch);
                hs.add(ch);
            }
        }
    }
}

```

=====

Problem: Remove Repeated Digits

Level: Medium

Source: zoho/Remove_Repeated_Digits.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```
import java.util.Scanner;

public class Remove_Repeated_Digits{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n1 = sc.nextInt();
        int n2 = sc.nextInt();

        for(int i=n1; i<=n2;i++){
            int last = i%10;
            int remain = i/10;
            if(last != remain){ System.out.print(i + " ");
            }
        }
    }
}
```

=====

Problem: Secnd Large

Level: Medium

Source: zoho/Secnd_Large.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```
import java.util.*;
public class Secnd_Large{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int Largest = arr[0];
        int Second = 0;

        for(int i=0;i<n;i++){
            if(arr[i] > Largest){
                Second = Largest;
                Largest = arr[i];
            }

            if(arr[i] > Second && arr[i] != Largest ){
                Second = arr[i];
            }
        }
        System.out.println("Largest: " + "" + Largest + "Second Largest " + Second);
    }
}
```

=====

Problem: Top K Frequent Elmnts

Level: Medium

Source: zoho/TopK_Frequent_Elmnts.java

Time complexity: $O(n \log k)$ in the common Top-K/heap approach

Space complexity: $O(n)$

Your original code:

```
import java.util.*;

public class TopK_Frequent_Elmnts {

    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] arr = new int[n];
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

int k = 2;

HashMap<Integer, Integer> map = new HashMap<>();

for (int num : arr) {
    if (map.containsKey(num)) {
        map.put(num, map.get(num) + 1);
    } else {
        map.put(num, 1);
    }
}

PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>((a, b) -> a.getValue() -
b.getValue());

for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " " + entry.getValue());
    pq.offer(entry);
    if (pq.size() > k) {
        pq.poll();
    }
}

while(!pq.isEmpty()){
    System.out.println(pq.poll().getKey());
}

}
}

```

=====

Problem: Two Sum

Level: Medium

Source: zoho/Two_Sum.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Two_Sum{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int target = sc.nextInt();
        boolean found = false;

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int i=0;i<n;i++){
            int res = target - arr[i];
            if(map.containsKey(res)){
                found = true;
                System.out.println(map.get(res) + " " + i);
            }
            else{
                map.put(arr[i],i);
            }
        }
    }
}

```

```

    }
}

// class Solution {
//     public int[] twoSum(int[] nums, int target) {
//         // your code
//         HashMap<Integer,Integer> map = new HashMap<>();

//         for(int i=0;i<nums.length;i++){
//             boolean found = false;
//             int res = target - nums[i];
//             if(map.containsKey(res)){
//                 found = true;
//                 return new int[]{map.get(res),i};
//             }else{
//                 map.put(nums[i],i);
//             }
//         }
//         return new int[]{};
//     }

//     public static void main(String[] args){
//         Scanner sc = new Scanner(System.in);
//         int n = sc.nextInt();
//         int[] arr = new int[n];
//         for(int i=0;i<n;i++){
//             arr[i] = sc.nextInt();
//         }

//         int target = sc.nextInt();
//         Solution obj = new Solution();
//         obj.twoSum(arr,target);
//     }
// }

```

=====

Problem: All Peak Element

Level: Medium

Source: zoho/arraylist/All_Peak_Element.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class All_Peak_Element{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        ArrayList<Integer> al = new ArrayList<>();

        for(int i=0;i<n;i++){
            boolean left_Check = (i == 0 || arr[i] > arr[i-1]);
            boolean right_check = (i == n-1 || arr[i] > arr[i+1]);

```

```

        if(left_Check && right_check){
            al.add(i);
        }
    }

    if(al.isEmpty()){
        System.out.println("Not found");
    }else{
        System.out.println(al);//returns peak element index.
    }

    for(int index : al){
        System.out.println("corresponding index " + arr[index]);//returns array value
    }
}
}

```

```

=====
Problem: First Non Repeating Char
Level: Medium
Source: zoho/arraylist/First_Non_Repeating_Char.java
Time complexity: O(n)
Space complexity: O(1) auxiliary space
Your original code:

```

```

import java.util.Scanner;

public class First_Non_Repeating_Char{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        String str = sc.nextLine();
        char[] ch = str.toCharArray();

        for(int i=0;i<str.length();i++){
            boolean found = false;
            for(int j=0;j<str.length();j++){
                if(i != j && ch[i] == ch[j]){
                    found = true;
                    break;
                }
            }
            if(!found){
                System.out.print(ch[i]);
                break;
            }
        }
    }
}

```

```

=====
Problem: Leaders in Array
Level: Medium
Source: zoho/arraylist/Leaders_in_Array.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;

public class Leaders_in_Array{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
    }
}

```

```

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        for(int i=0;i<n;i++){
            boolean leader = false;
            for(int j=1;j<n;j++){
                if(arr[i] > arr[j]){
                    leader = true;
                }
            }

            if(leader){
                System.out.println(arr[i]);
                return;
            }
        }
    }
}

```

=====

Problem: Majority Element

Level: Medium

Source: zoho/arraylist/Majority_Element.java

Time complexity: $O(n)$ average case

Space complexity: $O(n)$

Your original code:

```

import java.util.HashMap;
import java.util.Scanner;
public class Majority_Element{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int majority = n/2;

        // for(int i=0;i<n;i++){
        //     int count = 0;
        //     for(int j=0;j<n;j++){
        //         if(arr[i] == arr[j]){
        //             count++;
        //         }
        //     }
        //     if(count > majority){
        //         System.out.print(arr[i]);
        //         return;
        //     }
        // }

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int i=0;i<n;i++){
            if(map.containsKey(arr[i])){
                map.put(arr[i], map.getDefault(arr[i],0)+1);
            }else{
                map.put(arr[i],1);
            }

            if(map.get(arr[i]) > majority){
                System.out.print(arr[i]);
            }
        }
        // System.out.print(map + " ");
    }
}

```

```

    }
}

```

```

=====
Problem: Move Zeroes
Level: Medium
Source: zoho/arraylist/Move_Zeroes.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;

public class Move_Zeroes{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int j = 0;
        for(int i=0;i<n;i++){
            if(arr[i] != 0){
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                j++;
            }
        }

        for(int i=0;i<n;i++){
            System.out.print(arr[i] + " ");
        }
    }
}

```

```

=====
Problem: Peak Element
Level: Medium
Source: zoho/arraylist/Peak_Element.java
Time complexity: O(n)
Space complexity: O(n)
Your original code:

```

```

import java.util.Scanner;

public class Peak_Element {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int peak_index = -1;
        for (int i = 0; i < n; i++) {
            boolean leftCheck = (i == 0 || arr[i] > arr[i - 1]);
            boolean rightCheck = (i == n - 1 || arr[i] > arr[i + 1]);

```

```

        if (leftCheck && rightCheck) {
            peak_index = i;
            break;
        }
    }
    if (peak_index != -1) {
        System.out.println(peak_index);//returns index of value which is peak element(which is
greater than it neighbors)
    } else {
        System.out.println("No peak Elements found");
    }
}
}
}

```

=====

Problem: Peak Elemnt BS

Level: Medium

Source: zoho/arraylist/Peak_Elemnt_BS.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Peak_Elemnt_BS {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int peak_index = find_Peak(arr);

        System.out.println("index of peak element is" + " " + peak_index + " value is " + " " +
arr[peak_index]);
    }

    public static int find_Peak(int[] arr) {
        int low = 0;
        int high = arr.length - 1;

        while (low < high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] < arr[mid + 1]) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }
}

```

=====

Problem: Rotate Array

Level: Medium

Source: zoho/arraylist/Rotate_Array.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Rotate_Array{

```

```

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int[] arr = new int[n];

    for(int i=0;i<n;i++){
        arr[i] = sc.nextInt();
    }

    int k = sc.nextInt();

    k = k%n;//consider k = 2 & n = 4 if we divide k%n= we get reminder as k only

    for(int i=0;i<k;i++){
        int last = arr[n-1];

        for(int j=n-1;j>0;j--){
            arr[j] = arr[j-1];
        }
        arr[0] = last;
    }

    System.out.println("Rotated Array");
    for(int val : arr){
        System.out.print(val + " ");
    }
    System.out.println();
}
}

```

=====

Problem: Rotate Array by Reversal Alghthm

Level: Medium

Source: zoho/arraylist/Rotate_Array_byReversal_Alghthm.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

import java.util.*;

public class Rotate_Array_byReversal_Alghthm{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        int k = sc.nextInt()%n;

        reverse(arr, 0, n-1);
        reverse(arr, 0, k-1);
        reverse(arr, k, n-1);

        for(int val : arr){
            System.out.print(val + " ");
        }
    }

    public static void reverse(int[] arr, int start, int end){
        while(start < end){
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
}

```

```

    }
}

```

=====

Problem: Second Largest

Level: Medium

Source: zoho/arraylist/SecondLargest.java

Time complexity: $O(n)$

Space complexity: $O(1)$ auxiliary space

Your original code:

```

public class SecondLargest {

    public static int Solution(int[] a,int n){
        int largest = a[0];
        int secLarge = -1;
        for(int i=1;i<n;i++){
            if(a[i] > largest){
                secLarge = largest;
                largest = a[i];
            }
            if(a[i] < largest && a[i] > secLarge){
                secLarge = a[i];
            }
        }
        return secLarge;
    }
    public static void main(String[] args){

        int[] a = {6,6,6,6,6};
        int n = a.length;

        System.out.print(Solution(a,n));
    }
}

```

=====

Problem: Two sum using two pointers

Level: Medium

Source: zoho/arraylist/Two_sum_using_two_pointers.java

Time complexity: $O(n)$

Space complexity: $O(n)$

Your original code:

```

//works directly when array is sorted else we need to sort the elements by brute force else builtin
(Arrsy.sort)
import java.util.Scanner;

public class Two_sum_using_two_pointers{

    public void twoPointers(int[] nums,int target){

        int left = 0;
        int right = nums.length-1;
        while(left < right){
            if(nums[left] + nums[right] < target){
                left++;
            }else if(nums[left] + nums[right] > target){
                right--;
            }else{
                System.out.print(nums[left] + " " + nums[right]);
                return;
            }
        }
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
    }
}

```

```
    for(int i=0;i<n;i++){
        arr[i] = sc.nextInt();
    }

    int target = sc.nextInt();

    Two_sum_using_two_pointers tp = new Two_sum_using_two_pointers();

    tp.twoPointers(arr, target);
}
```